

# 오늘의 작업기록 — 2026년 7월 12일

## 예외 → HTTP 번역: 500이 400이 된 날

(서버가 "내 잘못"이라 거짓말하던 것을, "당신 요청이 무리였다"로 바로잡은 날)

### 한눈에

| 항목     | 내용                                                                                                          |
|--------|-------------------------------------------------------------------------------------------------------------|
| 목표     | 잔액 부족 예외가 <b>500</b> 으로 새는 것을 <b>4xx</b> 로 번역 (3회 이월된 과제)                                                   |
| 결과     | 완주 ✓ — before 500 → after <b>400</b> + 실패 사유 메시지, 정상 이체 <b>200</b> 회귀 확인                                    |
| 커밋     | <code>e6d10b5</code> "map InsufficientBalanceException to 400 (verified: 400 on shortfall, 200 on success)" |
| ★ 새 문법 | @ExceptionHandler(방아쇠 = URL이 아니라 <b>예외</b> ) · ResponseEntity(상태코드 + 본문을 함께 담는 그릇) · HttpStatus             |
| 판단     | 400(BAD_REQUEST) 채택 — 근거는 § 2-3                                                                             |
| 부수 실증  | 재부팅 후 a가 <b>2100</b> 으로 리셋 (7/7의 60이 증발) = <b>영속성 부재</b> , 8월 WAL의 이유를 눈으로 확인                               |
| 다음     | 없는 계좌(404) · amount≤0(400) 핸들러 — <b>같은 패턴이라 10분거리</b>                                                       |

7/7에 멍등성까지 검증했을 때 이미 알고 있었다 — 잔액이 부족하면 500이 나간다는 것. 그때는 "위치만 찍어두고" 지나갔고, 그 뒤 세 번(7/10, 7/11, 어제) 이월됐다. 오늘 닫았다. 코드는 9줄이었지만, 그 9줄이 API의 **정직성**을 바꿨다. 서버가 자기 잘못이 아닌 것을 자기 잘못이라 말하지 않게 됐다.

## 1. 병증 — 500은 무엇이 문제였나 (before)

서버를 띄우고 감당 못 할 금액을 요청했다 (a는 2100뿐인데 99999):

```
./gradlew bootRun > server.log 2>&1 &
grep "Started LedgerApi" server.log           # 판정: Started ... in 4.0 seconds

curl -i -X POST localhost:8080/transfer -H "Content-Type: application/json" \
  -d '{"fromId":"a","toId":"b","amount":99999,"idempotencyKey":"boom-1"}'
```

```
HTTP/1.1 500
{"timestamp":"2026-07-12T11:08:19.639Z","status":500,"error":"Internal Server Error","path":"/transfer"}
```

왜 이게 "틀린" 응답인가 — 상태코드는 "누구 탓인가"를 말한다

HTTP 상태코드는 단순한 번호가 아니라 **책임 소재의 선언**이다:

| 계열  | 의미           | 받는 쪽이 취할 행동                    |
|-----|--------------|--------------------------------|
| 2xx | 성공           | 결과를 쓴다                         |
| 4xx | 클라이언트(요청) 잘못 | 요청을 고쳐라. 그대로 재시도해봐야 소용없다       |
| 5xx | 서버 잘못        | 서버 문제이니 잠시 후 <b>재시도</b> 해볼 만하다 |

잔액 부족은 **a가 2100뿐인데 99999를 보내라고 한 요청**의 문제다. 서버는 정상 작동했고, 오히려 규칙(사잔 검증)에 따라 **정확하게 거절**했다. 그런데 500을 내보내면:

- 책임을 서버가 뒤집어쓴다** — "Internal Server Error" = 서버 내부가 고장났다는 뜻.
- 받는 쪽이 잘못 대응한다** — 앱 개발자는 5xx를 보면 "서버 일시 장애구나" 하고 재시도 로직을 돈다. 그런데 몇 번을 재시도해도 잔액은 그대로다. **무의미한 재시도**만 쌓인다.
- 이유를 안 알려준다** — 응답 어디에도 "왜" 실패했는지(잔액 부족)가 없다. 앱은 사용자에게 "알 수 없는 오류"라고 띄울 수밖에 없다.

→ 고쳐야 할 것 둘: **상태코드를 4xx로**, 그리고 **실패 사유를 본문에 담기**.

### 부수 실증 — a가 2100으로 리셋됐다 (영속성 부재)

7/7 검증 끝에 a는 **60**까지 깎여 있었다. 그런데 오늘 서버를 새로 띄우자 a는 다시 **2100**이다. 이유는 알고 있던 그대로다 — `private final Bank bank = new Bank();` 는 **서버 부팅 때 딱 한 번** 실행되고, 그 상태는 **메모리에만** 있다. 서버를 내리면 전부 증발하고, 다시 켜면 생성자가 처음부터 다시 돌아 계좌를 새로 만든다.

이것이 **영속성(persistence)의 부재**다. 진짜 원장은 깎다 쳐도 잔액이 남아야 한다. **8월 Layer 0(WAL)**이 정확히 이 문제 — "디스크에 먼저 기록해 죽어도 남기기" — 를 다룬다. 오늘 그 필요를 개념이 아니라 **숫자(60 → 2100)**로 봤다.

## 2. ★ 오늘의 새 문법 (플코드 + 정밀 해설)

## 2-1. 완성된 TransferController 전문 (커밋 e6d10b5)

```
package com.ledger.api;

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;

@RestController
public class TransferController {

    private final Bank bank = new Bank();

    public TransferController() {
        bank.createAccount("a", "이름", 2100);
        bank.createAccount("b", "이름", 3500);
        bank.createAccount("platform", "플랫폼", 0);
    }

    @PostMapping("/transfer")
    public String transfer(@RequestBody TransferRequest request) {
        bank.transfer(request.fromId(), request.toId(), request.amount(), request.idempotencyKey());
        return "a: " + bank.getAccount("a").getBalance()
            + ", b: " + bank.getAccount("b").getBalance()
            + ", platform: " + bank.getAccount("platform").getBalance();
    }

    // * 오늘 추가한 9줄
    @ExceptionHandler(InsufficientBalanceException.class)
    public ResponseEntity<String> handleInsufficientBalance(InsufficientBalanceException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("이체 실패: " + e.getMessage());
    }
}
```

## 2-2. @ExceptionHandler — 방아쇠가 URL이 아니라 "예외"인 애노테이션

이게 뭔가. 지금까지 본 매핑 애노테이션은 방아쇠가 URL이었다: - `@GetMapping("/health")` = "GET /health가 오면 이 메서드를 불러라" -

`@PostMapping("/transfer")` = "POST /transfer가 오면 이 메서드를 불러라"

`@ExceptionHandler` 는 방아쇠가 예외다: - `@ExceptionHandler(InsufficientBalanceException.class)` = "이 컨트롤러 안에서 `InsufficientBalanceException` 이 터지면, 500을 내보내지 말고 이 메서드를 대신 불러라"

어제(7/10) 판 원리가 그대로 적용된다. 부팅 시 스프링이 명찰 검사를 하며 `@RestController` 클래스를 훑을 때, `@GetMapping` / `@PostMapping` 명찰로 URL 표를 만들듯이 — `@ExceptionHandler` 명찰로는 "예외 → 처리 메서드" 표를 만든다. 이후 transfer 실행 중 예외가 튀어나오면, 스프링이 그 표를 뒤져 짝이 맞는 메서드를 호출한다. 짝이 없으면? 스프링의 기본 동작(= 500)으로 떨어진다. 지금까지 500이 나왔던 이유가 정확히 그것이다 — 표가 비어 있었다.

(`InsufficientBalanceException.class`) — 왜 `.class` 인가. 애노테이션 괄호에는 값이 들어간다( `@GetMapping("/health")` 의 문자열처럼). 여기서 넘기는 값은 "어떤 종류의 예외인가"라는 타입 정보이고, 자바에서 "타입 그 자체"를 값으로 넘길 때 쓰는 문법이 `타입이름.class` 다. 예외 객체(터진 그 물건)가 아니라 예외 종류(무엇이 터지면 반응할지)를 등록하는 것이라 이 형태가 된다.

매개변수 (`InsufficientBalanceException e`) — 여기 뭐가 들어오나. 실제로 터진 예외 객체가 들어온다. 스프링이 잡아서 넘겨준다. 그래서 `e.getMessage()` 로 예외에 담긴 메시지("잔액이 충분하지 않습니다")를 꺼내 응답 본문에 실을 수 있다 — mini-ledger에서 `catch (Exception e)` 로 잡아 `e.getMessage()` 를 찍던 것과 똑같은 객체다. 잡는 주체가 내 catch 블록에서 스프링으로 바뀌었을 뿐이다.

## 2-3. ResponseEntity — 상태코드까지 실어 보내는 그릇

지금까지의 return과 뭐가 다른가:

|      | <code>return String</code> (지금까지) | <code>return ResponseEntity&lt;String&gt;</code> (오늘)                          |
|------|-----------------------------------|--------------------------------------------------------------------------------|
| 담는 것 | 본문만                               | 상태코드 + 본문 (필요하면 헤더까지)                                                          |
| 상태코드 | 스프링이 알아서 (성공하면 200)               | 내가 지정                                                                          |
| 쓸 때  | <code>return "a: 1080...";</code> | <code>return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("...");</code> |

/health 나 /transfer 처럼 "성공하면 200"이 당연한 경우엔 String만 돌려줘도 스프링이 200을 붙여준다. 그러나 상태코드 자체가 메시지의 일부인 경우(오늘처럼 "이건 400이다"를 말해야 할 때)는 그릇이 필요하다. 그게 ResponseEntity다.

읽는 법: `ResponseEntity.status(...)` = 상태코드를 정하고 → `.body(...)` = 본문을 채워서 → 완성된 응답 객체를 반환. `HttpStatus` 는 상태코드를 숫자(400) 대신

이름(BAD\_REQUEST)으로 쓰게 해주는 목록이다 — 숫자를 외우지 않아도 되고, 오타가 컴파일 단계에서 잡힌다.

2-4. 400 vs 409 — 오늘의 판단

두 후보가 있었다: - **400 BAD\_REQUEST** — "요청이 잘못됐다"(클라이언트 책임 일반) - **409 CONFLICT** — "요청 형식은 맞지만 지금 자원의 상태와 충돌한다"

**400을 골랐다.** 근거: 클라이언트가 감당할 수 없는 금액을 요청한 것이고, 서버는 규칙대로 정상 거절했다. 즉 **요청 쪽 책임**이며, 받는 쪽이 취해야 할 행동("요청을 고쳐라, 재시도해봐야 소용없다")도 400의 의미와 정확히 일치한다. 409는 "동시 요청이 부딪혔다"처럼 **경합/충돌** 뉘앙스가 강해서, 단일 요청의 잔액 부족에는 과한 이름이다. 단순하고 정확한 쪽을 택했다.

(더 정밀하게 가면 402 Payment Required 같은 것도 있지만 실무에서 거의 안 쓰이고, 결제 API들도 대개 400/422 계열로 처리한다. 지금 단계에선 400이 맞다.)

3. 검증 — before / after (m105 다음 단계의 완료 조건)

서버 재시작 후 (코드를 바꿨으므로 필수):

```
kill %1
./gradlew bootRun > server.log 2>&1 &
sleep 18 && grep "Started LedgerApi\|error:" server.log
# 판정: Started LedgerApiApplication in 4.044 seconds   (컴파일 에러 없음)
```

| # | 요청                       | before (오늘 아침)                    | after (핸들러 적용 후)                                     |
|---|--------------------------|-----------------------------------|------------------------------------------------------|
| ① | 잔액 부족 (a=2100, 99999 이체) | 500 Internal Server Error (이유 없음) | 400 + 이체 실패: 잔액이 충분하지 않습니다                           |
| ② | 정상 이체 (1000)             | 200 + 잔액                          | 200 + a: 1080, b: 4500, platform: 20 (변화 없음 = 회귀 없음) |

실제 출력:

```
HTTP/1.1 400
Content-Type: text/plain; charset=UTF-8
Content-Length: 50

이체 실패: 잔액이 충분하지 않습니다
```

```
HTTP/1.1 200
a: 1080, b: 4500, platform: 20
```

②(정상 이체 200)를 반드시 확인하는 이유 — 회귀 테스트. 새 코드를 넣었을 때 "새 기능이 작동하나"만 보면 절반이다. **원래 되던 것이 여전히 되나**도 봐야 한다. 예외 핸들러가 잘못 걸리면 멀쩡한 이체까지 400으로 막힐 수 있는데, 200이 그대로 나온 것이 그런 일이 없음을 증명한다. (그리고 잔액이 정확히 1080 / 4500 / 20 = 이체 1000 + 수수료 20이 반영된 값이라, 도메인 로직도 무사하다.)

서버 내림 (공개 IP 규칙):

```
kill %2
ss -tlnp | grep 8080           # 판정: 빈 출력 = 포트 닫힘
```

7/7과 동일하게, **kill** 직후 ss에는 자식 java(pid 1303227)가 잠시 남아 보였다. 부모(gradle)가 죽으면 자식은 **시차를 두고** 정리된다 — 판정 기준은 언제나 **ss의 빈 출력**이다.

커밋:

```
git add . && git commit -m "map InsufficientBalanceException to 400 (verified: 400 on shortfall, 200 on success)" && git push
# 판정: [main e6d10b5] ... 1 file changed, 9 insertions(+)
#      7c7b7da..e6d10b5  main -> main
```

4. 트러블 — 서버가 뜨기 전에 curl을 쳤다

증상: 부팅 직후 **grep "Started LedgerApi"** 가 빈 출력이었는데, 그대로 curl을 쳐서:

```
curl: (7) Failed to connect to localhost port 8080 after 0 ms: Couldn't connect to server
```

원인: 서버가 아직 뜨는 중(Gradle 빌드 포함이라 10~20초). 포트가 안 열렸으니 **연결 자체가 거부된 것**.

교훈 — 세 가지 실패는 완전히 다른 것이다:

| 증상                | 뜻                    | 원인                       |
|-------------------|----------------------|--------------------------|
| Failed to connect | 서버가 없다               | 안 떴거나 이미 내려감             |
| 404 Not Found     | 서버는 있는데 그 경로가 없다     | 컨트롤러 미등록/파일 부재 (7/7의 그것) |
| 500               | 서버·경로 다 있는데 처리 중 터졌다 | 오늘의 그것                   |

→ 판정 순서를 지킨다: `grep "Started"` 확인 → 그 다음에 `curl`. 성급하게 친 `curl`은 정보를 주지 않는다.

## 5. 로드맵 위치와 다음

- **ledger-api 현재 모습:** `GET /health` · `POST /transfer` (역등성 + 복식부기 검증 + **정확한 오류 응답**). 도메인은 여전히 mini-ledger에서 수정 0줄로 가져온 그대로.
- **다음(짧음, 10분거리):** 같은 패턴으로 핸들러 2개 추가 — **없는 계좌 → 404**( `AccountNotFoundException` ), **amount ≤ 0 → 400**. 오늘 패턴을 그대로 복제하면 된다.
- **8월 Layer 0(WAL):** 오늘 재부팅으로 잔액이 초기화되는 것을 눈으로 봤다(60 → 2100). 영속성 부재 + 7/5의 원자성 한계, 두 실이 WAL 한 지점으로 수렴한다.

## 6. 검증 질문 (답 적지 말 것)

1. 4xx와 5xx는 각각 "누구 탓"인가? 받는 쪽이 취하는 행동이 어떻게 달라지나?
2. 잔액 부족이 500이면 안 되는 이유 셋은?
3. `@ExceptionHandler` 의 방아쇠는 무엇인가? `@PostMapping` 과 무엇이 같고 무엇이 다른가?
4. 어제 배운 "명찰 검사" 원리로, 스프링이 예외 핸들러를 찾아내는 과정을 설명하라. 표가 비어 있으면 어떻게 되나?
5. `(InsufficientBalanceException.class)` 에서 `.class` 가 붙는 이유는? 메서드 매개변수 `e` 에는 뭐가 들어오나?
6. `return String` 과 `return ResponseEntity<String>` 의 차이는? 언제 후자가 필요한가?
7. 400과 409 중 400을 고른 근거는?
8. 정상 이체 200을 굳이 다시 확인한 이유는? (회귀)
9. `Failed to connect` / `404` / `500` — 세 실패의 원인은 각각 무엇인가?
0. 재부팅 후 a가 2100인 이유는? 이것이 8월 계획과 어떻게 연결되나?

## 7. 회고

세 번 이월된 과제였다. 7/7에 "위치만 찍어두고" 넘긴 500을, 7/10엔 애노테이션 개념에 붙들려서, 7/11엔 시간이 없어서 못 달았다. 오늘 30분으로 달았고 — 실제 코드는 **9줄** 이었다. 미룬 시간이 코드 분량보다 훨씬 길었던 셈이다. 이월의 비용은 작업량이 아니라 **재진입 비용**이라는 걸 배웠다. 매번 다시 서버 띄우고, 다시 맥락 복원하고. 짧은 과제일수록 그 자리에서 끝내는 게 싸다.

그래도 7/10에 애노테이션을 판 것이 오늘 값을 했다. `@ExceptionHandler` 를 보는 순간 "아, 방아쇠가 URL 대신 예외인 명찰이구나"가 바로 잡혔다. 어제 개념을 안 찢으면 오늘은 복붙이었을 거고, "왜 되는지 모르는 코드"가 하나 더 늘었을 것이다. 개념을 판 날은 진도가 0인 것처럼 보이지만, 그 다음 날의 코드가 이해된 코드가 된다.

그리고 이 9줄이 바꾼 것이 생각보다 크다. 이전의 API는 잔액이 부족할 때 **거짓말**을 했다 — "서버 내부 오류"라고. 클라이언트는 그 말을 믿고 무의미한 재시도를 했을 것이다. 이제는 "당신 요청이 무리였고, 이유는 잔액 부족"이라고 정확히 말한다. 기능이 늘어난 게 아니라 **정직해진 것**이다. 결제 API에서 이건 사소한 차이가 아니다 — 오류 응답이 부정확하면, 그걸 받는 쪽의 모든 대응이 어긋난다.

마지막으로 오늘 숫자 하나를 봤다: a가 60에서 2100으로 돌아와 있었다. 며칠 전 문답집에서 "영속성이 없다"를 글로 썼는데, 오늘은 그게 눈앞의 숫자였다. 8월에 WAL을 만들 이유가 이제 문장이 아니라 장면으로 남았다.

**한 줄:** 9줄로 500을 400으로 — 서버가 자기 잘못이 아닌 것을 자기 잘못이라 말하지 않게 됐다. 세 번 이월된 과제, 오늘 닫음.